

Atividade: Modularização de Código e Análise Comparativa de LLMs

Diogo Estéfano Cambraia Freitas | Engenharia de Telecomunicações

Pseudocódigo

O programa receberá os inputs de leituras de potência de sinais recebidas em dBm. Deverá haver uma lista dinâmica que armazena a série de leituras.

Valores de leitura que não estão no intervalo de -120 a 0 ou não numéricos são descartados.

Haverá uma função que torna os valores que não estiverem na escala dBm devem ser convertidos para tal.

Haverá uma função que calcula as seguintes estatísticas: média, mínimo, máximo, desvio padrão e quantidade de amostras válidas vs. descartadas.

Haverá uma função que classifica todas as medições nos seguintes limiares: limiar alerta= menor que -85 e limiar crítico = menor que -100. Caso não seja nenhum desses casos, é classificado como normal.

Haverá uma função que contabiliza quantas medições ultrapassaram o limiar crítico, e identifica a maior sequência consecutiva que teve.

Haverá uma função que com o resultado de todo esse processamento das funções printa um relatório com tudo encontrado.

Prompt

Crie o código python para o programa com o pseudocódigo abaixo. Foque em modularização e separação das funções. Além do mais, crie um outro arquivo .py simples que gere x valores aleatórios de medições factíveis de dBm de potência recebida. X é o input de quantos valores quero.

Pseudocódigo do primeiro programa:

“

O programa receberá os inputs de leituras de potência de sinais recebidas em dBm. Deverá haver uma lista dinâmica que armazena a série de leituras.

Valores de leitura que não estão no intervalo de -120 a 0 ou não numéricos são descartados.

Haverá uma função que torna os valores que não estiverem na escala dBm devem ser convertidos para tal.

Haverá uma função que calcula as seguintes estatísticas: média, mínimo, máximo, desvio padrão e quantidade de amostras válidas vs. descartadas.

Haverá uma função que classifica todas as medições nos seguintes limiares: limiar alerta= menor que -85 e limiar crítico = menor que -100. Caso não seja nenhum desses casos, é classificado como normal.

Haverá uma função que contabiliza quantas medições ultrapassaram o limiar crítico, e identifica a maior sequência consecutiva que teve.

Haverá uma função que com o resultado de todo esse processamento das funções printa um relatório com tudo encontrado.“

LLM 1 | Claude AI

Programa principal

```
"""
```

```
Analizador de Potência de Sinal em dBm
Analisa leituras de potência recebida, calcula estatísticas,
classifica limiares e identifica sequências críticas.
```

```
"""
```

```
import math
```

```
# -----
```

```
# Constantes
```

```
# -----
```

```
DBM_MIN = -120
```

```
DBM_MAX = 0
```

```
LIMIAR_ALERTA = -85
```

```
LIMIAR_CRITICO = -100
```

```
# -----
```

```
# 1. Validação e conversão de entradas
```

```
# -----
```

```
def tentar_convertir_para_float(valor) -> float | None:
```

```
    """Tenta converter um valor para float. Retorna None se não for possível."""
```

```
    try:
```

```
        return float(valor)
```

```
    except (ValueError, TypeError):
```

```
        return None
```

```
def converter_para_dbm(valor: float) -> float:
```

```
    """
```

```
    Converte para dBm caso o valor esteja em Watts (positivo e > 1e-20).
```

```
    Valores já negativos ou zero são assumidos como já sendo dBm.
```

```
    """
```

```
    if valor > 0:
```

```
        # Assume que é potência em Watts — converte para dBm
```

```
        return 10 * math.log10(valor * 1000)
```

```
    return valor # já está em dBm
```

```
def validar_dbm(valor: float) -> bool:
```

```
    """Verifica se o valor (após conversão) está no intervalo válido de dBm."""
```

```
    return DBM_MIN <= valor <= DBM_MAX
```

```
def processar_leituras(leituras_brutas: list) -> tuple[list[float], list]:
```

```
    """
```

```
    Processa uma lista de leituras brutas:
```

- Tenta conversão numérica
- Converte para dBm se necessário
- Valida o intervalo [-120, 0]

```
    Retorna:
```

```
    validas — lista de floats em dBm aceitos
```

```
    descartados — lista dos valores originais rejeitados
```

```
    """
```

```
    validas: list[float] = []
```

```
    descartados: list = []
```

```
    for entrada in leituras_brutas:
```

```
        valor = tentar_convertir_para_float(entrada)
```

```
        if valor is None:
```

```
            descartados.append(entrada)
```

```
            continue
```

```
        valor_dbm = converter_para_dbm(valor)
```

```
        if validar_dbm(valor_dbm):
```

```
            validas.append(valor_dbm)
```

```
        else:
```

```
            descartados.append(entrada)
```

```
    return validas, descartados
```

```
# _____
```

```
# 2. Estatísticas
```

```
# _____
```

```
def calcular_estatisticas(validas: list[float], descartados: list) -> dict:
```

```
    """
```

```
    Calcula: média, mínimo, máximo, desvio padrão,  
    quantidade de amostras válidas e descartadas.
```

```
    """
```

```
    n = len(validas)
```

```
    if n == 0:
```

```
        return {
```

```
            "media": None,
```

```
            "minimo": None,
```

```
            "maximo": None,
```

```
            "desvio_padrao": None,
```

```
            "qtd_validas": 0,
```

```
            "qtd_descartadas": len(descartados),
```

```

    }

    media = sum(validas) / n
    minimo = min(validas)
    maximo = max(validas)
    variancia = sum((x - media) ** 2 for x in validas) / n
    desvio_padrao = math.sqrt(variancia)

    return {
        "media": media,
        "minimo": minimo,
        "maximo": maximo,
        "desvio_padrao": desvio_padrao,
        "qtd_validas": n,
        "qtd_descartadas": len(descartados),
    }

```

```

# -----
# 3. Classificação por limiar
# -----

```

```

def classificar_medicao(valor_dbm: float) -> str:
    """Classifica uma medição individual conforme os limiares definidos."""
    if valor_dbm < LIMIAR_CRITICO:
        return "crítico"
    if valor_dbm < LIMIAR_ALERTA:
        return "alerta"
    return "normal"

```

```

def classificar_todas(validas: list[float]) -> list[dict]:
    """
    Retorna uma lista de dicionários com valor e classificação
    para cada medição válida.
    """
    return [
        {"valor": v, "classificacao": classificar_medicao(v)}
        for v in validas
    ]

```

```

# -----
# 4. Análise de sequências críticas
# -----

```

```

def analisar_criticos(classificacoes: list[dict]) -> dict:
    """
    Contabiliza medições críticas e identifica
    a maior sequência consecutiva de críticos.
    """
    total_criticos = 0

```

```

maior_sequencia = 0
sequencia_atual = 0

for item in classificacoes:
    if item["classificacao"] == "crítico":
        total_criticos += 1
        sequencia_atual += 1
        maior_sequencia = max(maior_sequencia, sequencia_atual)
    else:
        sequencia_atual = 0

return {
    "total_criticos": total_criticos,
    "maior_sequencia_consecutiva": maior_sequencia,
}

```

```

# —————
# 5. Relatório
# —————

```

```

def _linha(char: str = "—", largura: int = 55) -> str:
    return char * largura

```

```

def imprimir_relatorio(
    estatisticas: dict,
    classificacoes: list[dict],
    analise_criticos: dict,
) -> None:
    """Imprime o relatório completo de análise de sinal."""

    sep = _linha()
    sep2 = _linha("=")

    print(f"\n{sep2}")
    print("    RELATÓRIO DE ANÁLISE DE SINAL (dBm)")
    print(sep2)

    # — Amostras —
    print("\n📊 AMOSTRAS")
    print(sep)
    print(f"  Válidas   : {estatisticas['qtd_validas']}")
    print(f"  Descartadas: {estatisticas['qtd_descartadas']}")

    # — Estatísticas —
    print(f"\n📈 ESTATÍSTICAS")
    print(sep)
    if estatisticas["media"] is None:
        print("  Sem amostras válidas para calcular estatísticas.")
    else:
        print(f"  Média      : {estatisticas['media']:.2f} dBm")

```

```

print(f" Mínimo      : {estatisticas['minimo']:.2f} dBm")
print(f" Máximo      : {estatisticas['maximo']:.2f} dBm")
print(f" Desvio Padrão: {estatisticas['desvio_padrao']:.2f} dBm")

# — Classificações —
print(f"\n🔍 CLASSIFICAÇÃO DAS MEDIÇÕES")
print(sep)
contagem = {"normal": 0, "alerta": 0, "crítico": 0}
for item in classificacoes:
    contagem[item["classificacao"]] += 1

total = len(classificacoes)
for cat, qtd in contagem.items():
    pct = (qtd / total * 100) if total > 0 else 0
    icone = {"normal": "✅", "alerta": "⚠️", "crítico": "🔴"}[cat]
    print(f" {icone} {cat.capitalize():10s}: {qtd:4d} ({pct:5.1f}%)")

print(f"\n Limiares de referência:")
print(f" Alerta → abaixo de {LIMIAR_ALERTA} dBm")
print(f" Crítico → abaixo de {LIMIAR_CRITICO} dBm")

# — Análise de críticos —
print(f"\n🔴 ANÁLISE DE EVENTOS CRÍTICOS")
print(sep)
print(f" Total de medições críticas      : {analise_criticos['total_criticos']}")
print(f" Maior sequência consecutiva crítica: {analise_criticos['maior_sequencia_consecutiva']}")

# — Lista detalhada —
print(f"\n📋 DETALHAMENTO (primeiras 20 amostras)")
print(sep)
print(f" {'#':>4} {'Valor (dBm)':>12} {'Classificação'}")
print(f" {'-'*4} {'-'*12} {'-'*14}")
for i, item in enumerate(classificacoes[:20], start=1):
    icone = {"normal": "✅", "alerta": "⚠️", "crítico": "🔴"}[item["classificacao"]]
    print(f" {i:>4} {item['valor']:>11.2f} {icone} {item['classificacao']}")
if len(classificacoes) > 20:
    print(f" ... ({len(classificacoes) - 20} amostras omitidas)")

print(f"\n{sep*2}\n")

```

```

# _____
# 6. Pipeline principal
# _____

```

```

def executar_analise(leituras_brutas: list) -> None:
    """
    Orquestra todo o pipeline: validação → estatísticas
    → classificação → análise crítica → relatório.
    """
    validas, descartados = processar_leituras(leituras_brutas)
    estatisticas = calcular_estatisticas(validas, descartados)

```

```
classificacoes = classificar_todas(validas)
analise_criticos = analisar_criticos(classificacoes)
imprimir_relatorio(estatisticas, classificacoes, analise_criticos)
```

```
# -----
# 7. Entrada interativa
# -----
```

```
def coletar_leituras_interativo() -> list:
    """
    Coleta leituras do usuário via terminal.
    Digite 'fim' para encerrar a entrada.
    """
    print("\n=== Entrada de Leituras de Potência ===")
    print("Digite os valores em dBm (ou Watts).")
    print("Digite 'fim' para encerrar.\n")

    leituras = []
    while True:
        entrada = input("Leitura: ").strip()
        if entrada.lower() == "fim":
            break
        leituras.append(entrada)

    return leituras
```

```
# -----
# Execução
# -----
```

```
if __name__ == "__main__":
    leituras = coletar_leituras_interativo()
    executar_analise(leituras)
```

Gerador de valores

```
"""
Gerador de Leituras Aleatórias de Potência em dBm
Gera X valores simulando medições reais de sinal recebido.
```

```
Distribuição realista:
- 60% valores normais (-85 a 0 dBm)
- 25% valores de alerta (-100 a -85 dBm)
- 15% valores críticos (-120 a -100 dBm)
"""
```

```
import random
```

```
DBM_MIN = -120
DBM_MAX = 0
```

```

FAIXAS = [
    (0.60, -85, 0), # normal
    (0.25, -100, -85), # alerta
    (0.15, -120, -100), # crítico
]

```

```

def gerar_valor_dbm() -> float:
    """Gera um único valor aleatório de dBm com distribuição realista."""
    sorteio = random.random()
    acumulado = 0.0
    for peso, minimo, maximo in FAIXAS:
        acumulado += peso
        if sorteio < acumulado:
            return round(random.uniform(minimo, maximo), 2)
    return round(random.uniform(DBM_MIN, DBM_MAX), 2)

```

```

def gerar_leituras(quantidade: int) -> list[float]:
    """Gera uma lista com `quantidade` valores de dBm."""
    return [gerar_valor_dbm() for _ in range(quantidade)]

```

```

def main() -> None:
    print("=== Gerador de Leituras de Potência (dBm) ===\n")

    while True:
        entrada = input("Quantos valores deseja gerar? ").strip()
        try:
            quantidade = int(entrada)
            if quantidade <= 0:
                print(" ⚠ Digite um número inteiro positivo.\n")
                continue
            break
        except ValueError:
            print(" ⚠ Entrada inválida. Digite um número inteiro.\n")

    leituras = gerar_leituras(quantidade)

    print(f"\n{quantidade} valor(es) gerado(s):\n")
    for i, v in enumerate(leituras, start=1):
        print(f" {v:>8.2f}")

    # Salva em arquivo de texto para uso externo
    nome_arquivo = "leituras_geradas.txt"
    with open(nome_arquivo, "w") as f:
        for v in leituras:
            f.write(f"{v}\n")

    print(f"\n✅ Valores salvos em '{nome_arquivo}'")
    print(" (Use esse arquivo como entrada para o analisador_sinal.py)\n")

```



```
if __name__ == "__main__":  
    main()
```

LLM 2 | Chat GPT

Programa principal

```
from statistics import mean, stdev
```

```
# =====  
# Conversão e validação  
# =====
```

```
def converter_para_dbm(valor):  
    """  
    Converte valores para dBm caso necessário.  
    Atualmente assume que o valor já está em dBm.  
    Adapte esta função para outros formatos (mW, W, etc.).  
    """  
    return valor
```

```
def validar_leitura(valor):  
    """  
    Verifica se o valor é numérico e está no intervalo válido.  
    """  
    try:  
        valor = float(valor)  
    except (ValueError, TypeError):  
        return False  
  
    return -120 <= valor <= 0
```

```
# =====  
# Processamento das leituras  
# =====
```

```
def processar_leituras(leituras):  
    """  
    Converte e filtra leituras inválidas.  
    """  
    validas = []  
    descartadas = 0  
  
    for leitura in leituras:  
        try:  
            valor = converter_para_dbm(float(leitura))  
  
            if validar_leitura(valor):  
                validas.append(valor)
```

```

        else:
            descartadas += 1

    except (ValueError, TypeError):
        descartadas += 1

    return validas, descartadas

# =====
# Estatísticas
# =====

def calcular_estatisticas(leituras_validas, descartadas):
    """
    Calcula estatísticas básicas.
    """
    if not leituras_validas:
        return {
            "media": None,
            "minimo": None,
            "maximo": None,
            "desvio_padrao": None,
            "validas": 0,
            "descartadas": descartadas
        }

    return {
        "media": mean(leituras_validas),
        "minimo": min(leituras_validas),
        "maximo": max(leituras_validas),
        "desvio_padrao": (
            stdev(leituras_validas)
            if len(leituras_validas) > 1
            else 0
        ),
        "validas": len(leituras_validas),
        "descartadas": descartadas
    }

# =====
# Classificação
# =====

def classificar_medicao(valor):
    """
    Classifica uma única medição.
    """
    if valor < -100:
        return "CRITICO"

```

```

    if valor < -85:
        return "ALERTA"

    return "NORMAL"

def classificar_todas(leituras):
    """
    Retorna lista de classificações.
    """
    return [(valor, classificar_medicao(valor))
            for valor in leituras]

# =====
# Análise de limiar crítico
# =====

def analisar_criticos(leituras):
    """
    Conta quantas medições ficaram abaixo do limiar crítico
    e encontra a maior sequência consecutiva.
    """
    total_criticos = 0
    maior_sequencia = 0
    sequencia_atual = 0

    for valor in leituras:

        if valor < -100:
            total_criticos += 1
            sequencia_atual += 1

        if sequencia_atual > maior_sequencia:
            maior_sequencia = sequencia_atual

        else:
            sequencia_atual = 0

    return total_criticos, maior_sequencia

# =====
# Relatório
# =====

def imprimir_relatorio(
    estatisticas,
    classificacoes,
    total_criticos,
    maior_sequencia
):

```

```
"""
```

```
Exibe relatório final.
```

```
"""
```

```
print("\n" + "=" * 50)
print("RELATÓRIO DE POTÊNCIA RECEBIDA")
print("=" * 50)

print("\nESTATÍSTICAS")
print(f"Média: {estatisticas['media']:.2f} dBm"
      if estatisticas['media'] is not None
      else "Média: N/A")

print(f"Mínimo: {estatisticas['minimo']} dBm")
print(f"Máximo: {estatisticas['maximo']} dBm")
print(f"Desvio padrão: {estatisticas['desvio_padrao']:.2f} dB"
      if estatisticas['desvio_padrao'] is not None
      else "Desvio padrão: N/A")

print(f"Válidas: {estatisticas['validas']}")
print(f"Descartadas: {estatisticas['descartadas']}")

print("\nCLASSIFICAÇÕES")

for valor, classe in classificacoes:
    print(f"{valor:.2f} dBm -> {classe}")

print("\nANÁLISE DE CRÍTICOS")
print(f"Quantidade crítica: {total_criticos}")
print(f"Maior sequência crítica consecutiva: {maior_sequencia}")

print("=" * 50)
```

```
# =====
# Programa principal
# =====
```

```
def main():
```

```
    leituras = []
```

```
    print("Digite valores de potência (dBm).")
    print("Digite 'fim' para encerrar.\n")
```

```
    while True:
```

```
        entrada = input("Leitura: ")
```

```
        if entrada.lower() == "fim":
            break
```

```

        leituras.append(entrada)

leituras_validas, descartadas = processar_leituras(leituras)

estatisticas = calcular_estatisticas(
    leituras_validas,
    descartadas
)

classificacoes = classificar_todas(
    leituras_validas
)

total_criticos, maior_sequencia = analisar_criticos(
    leituras_validas
)

imprimir_relatorio(
    estatisticas,
    classificacoes,
    total_criticos,
    maior_sequencia
)

if __name__ == "__main__":
    main()

```

Gerador de valores

```

import random

def gerar_medicoes(quantidade):
    """
    Gera medições realistas de RSSI em dBm.
    Faixa típica:
    -110 dBm (muito ruim)
    -40 dBm (excelente)
    """

    return [
        round(random.uniform(-110, -40), 2)
        for _ in range(quantidade)
    ]

def main():

    quantidade = int(
        input("Quantas medições deseja gerar? ")
    )

```

```

medicoes = gerar_medicoes(quantidade)

print("\nMedições geradas:")

for valor in medicoes:
    print(valor)

if __name__ == "__main__":
    main()

```

Tabela Comparativa

#	Critério de modularização	Claude AI	Chat GPT
1	Decomposição em funções, cada uma com responsabilidade única	2	2
2	Nomes descritivos de funções e variáveis	2	2
3	Ausência de duplicação de código (DRY)	2	2
4	Parâmetros e retornos bem definidos; evita variáveis globais	2	2
5	Reaproveitamento de funções (ex.: validação usada em um só lugar)	2	2
6	Existência de uma função principal organizando o fluxo	2	1
7	Legibilidade geral e comentários úteis (sem excesso)	2	1
	Total (máx. 7)		

Análise crítica

Novamente me pareceu que o código gerado pela LLM Claude AI é mais formal e pronto para ser já ser aplicado. O do Chat GPT me pareceu mais fraco com suas funções. Com isso, o resultado da Claude parece tratar mais casos de exceções e ter um processamento melhor, enquanto o código gerado pelo Chat GPT está mais simples (o que com certeza não significa pior, mas menos elaborado.).

Perguntei a Claude sua opinião comparando o próprio código com o do Chat GPT, e a mesma disse “O meu código é mais completo visualmente e funcionalmente (conversão de Watts, ícones, percentuais, reuso de classificações), enquanto o do documento é mais simples e direto”.